K215: Direct Seed Models and Shared Propositional Proofs

Two checked components for simplifying the Rocq and Lean developments

Research assistance for Vladimir Reshetnikov
OpenAI

October 21, 2026 (America/Los_Angeles)

Abstract

The current K215 development already contains many of the obvious proof reductions. This report develops two specific next steps against the pinned repository: a direct Lean construction of a full seed model, and a shared Rocq propositional proof core. The direct construction implements an earlier uncompiled repository proposal and aligns Lean with a construction already used by the Rocq lower bound. Its essential point is that the completeness interface supplies truth of original named formulas with literal equality. One can therefore build the seed model on the original domain, using an interpreted formula code as a fresh private name, and bypass the preliminary indiscernibility quotient. The Rocq refactor packages 71 duplicated derivations once over implication, negation, modus ponens, and the three existing propositional axiom rules. Concrete adapters preserve both sets of exported names. The archive contains source overlays, reproduction programs, exact input identities, compiler evidence, and independent review. The status and limits of compilation are stated separately for the semantic components and the complete physical-machine endpoint. No Turing-machine transition is changed.

Contents

1 Recommendation and scope **3**
1.1 Precisely which repository state was reviewed . . . 3
1.2 The theorem being preserved . . . 3
1.3 Measured result and verification boundary . . . 4

2 What recent work has already accomplished **4**

3 The Lean opportunity: use the original model **5**
3.1 The current lower chain . . . 5
3.2 The decisive completeness interface . . . 5
3.3 The seed property . . . 6

4 A direct full-seed theorem **6**
4.1 A fresh name supplied by the interpreted code . . . 6
4.2 Named bundle semantics . . . 7
4.3 Derivability of the named bundle . . . 7

5 Integration into the Lean lower bridge **8**
5.1 What can now be removed from this proof path . . . 8
5.2 A small interface is essential . . . 8
5.3 The endpoint substitution . . . 9

6 The Rocq opportunity: one propositional derivation library **9**
6.1 The duplicated fragment . . . 9
6.2 The minimal interface . . . 9
6.3 Why the abstraction is sound . . . 9
6.4 The real size tradeoff . . . 10

7 Verification, reproduction, and adoption **10**
7.1 Evidence is tied to source identities . . . 10
7.2 Lean verification . . . 10
7.3 Rocq component verification . . . 11
7.4 Independent review and limits of performance claims . . . 11
7.5 Suggested integration order . . . 11

8 What should not be simplified away **12**

9 Further research questions **12**

10 Sources and attribution **13**

A Archive map and reproducibility **14**

1 Recommendation and scope

The recommended next integration is *direct seed construction in Lean plus one shared propositional core in Rocq*. These remove different kinds of unnecessary work: an intermediate model construction in the first case, and duplicate derivation bodies in the second. They preserve the target theorem and the existing calculus. They do not search for a different machine or rely on a smaller machine to simplify the proof.

1.1 Precisely which repository state was reviewed

The repository is https://github.com/VladimirReshetnikov/ProveIt. All source comparisons in this report use commit `6d9800d1ca67c52db0eb6e1c65df7b5cab9cdb95`, whose latest relevant change generalizes the K217 enumerator to arbitrary states between candidates. The timestamp is October 22 in UTC and October 21 in the user's local time zone. The previous changes already include the phase-fourteen K215 checkpoint and the proposed CP/GIC-to-QHyp quotient adapter.

The independent source census reproduces the phase-fourteen Lean endpoint exactly: **178 project modules, 2,093,112 source bytes, and 40,517 physical source lines**. This is an import-closure measurement; it is not a count of mathematical lemmas or a measurement of proof-object size. The source identity and complete module list are retained in `evidence/baseline-lean-closure.json`.

The dense machine table has SHA-256

`fdbcef0825886035c3091c8537a69535bc96f9e265f0ae45863543d40ae70090.`

It has 215 working states and a separate halt. Neither proposal changes that table.

1.2 The theorem being preserved

The maintained Lean endpoint has type

```
BlankHalts machine215 <-> BProv ZFCax_s [] Form.fBot
```

and its nonhalting corollary negates both sides. The Rocq endpoint similarly identifies halting of the exact dense table with a finite classical refutation of its explicitly defined ZFC theory. These are external proof-assistant theorems about the machine and syntactic consistency. The repository also explains why the ordinary mathematical argument can be formalized in ZFC. The present changes do not supply a serialized derivation in ZFC's own proof syntax, and they do not close the separately open internal PA endpoint.

The distinction matters: proving a new component, checking a source overlay, and rebuilding the entire physical endpoint are three different achievements. We use each description only for its corresponding evidence.

1.3 Measured result and verification boundary

Measured source scope	Before	After	Change
Lean K215 import closure	178 modules	177 modules	One fewer module
Same closure, bytes	2,093,112	2,086,255	6,857 fewer
Same closure, physical lines	40,517	40,387	130 fewer
Rocq paired propositional sources, bytes	23,160	16,001	7,159 fewer
Same sources, physical lines	297	261	36 fewer
Maintained Rocq derivation bodies	142	71	One copy per lemma

The Lean figure measures the *active endpoint closure*. Keeping the old quotient interfaces available adds 7,472 bytes and 184 lines across the six changed or new production files overall; compatibility preservation therefore does not shrink the entire repository. The three modules leaving this endpoint closure are `Code254Models`, `KSeedModel222`, and `KIndisc222`; two small interface/direct-proof modules enter it. The numerical seed semantics module remains because the upper chain still uses it.

The checked scope is the new Lean semantic component and its recorded clients, and the Rocq propositional component with its actual consumers. The entire literal K215 endpoint has not been rebuilt in this packet. The repository's existing full-endpoint checks remain evidence for the original pinned sources; the changed endpoint requires the stated integration gate.

The reductions are intentionally reported with their costs. Adding a shared abstraction is useful only when its consumers actually share it. Likewise, removing a quotient from an informal diagram does not remove it from the compiled dependency closure until imports and shared declarations are separated.

2 What recent work has already accomplished

This work begins after substantial simplification. Treating earlier results as new would obscure the remaining problems. The current source and proposal index establish the following context; references are given in Section 10.

Existing work	Consequence for the present recommendation
Shared implication core and named derived rules	The numerical dominated implication argument and the two Lean named calculi already share major parts of their derivations. The Rocq target below is a separate pair of classical propositional proof files.
Transitive-container factoring	Phase fourteen removes a duplicate profile development and its generator. We use that checkpoint as the baseline.
Checked macro words	The Lean AX construction for the permuted source is already a checked word; the dominated source now uses the same template. Recommending that transformation again would add little.
Source and runtime certificates	The proposals already cover trailing-zero normalization, balanced lookup, affine cores, semantic site contracts, uniform decoder prefixes, and positive simulations. They retain their own adoption and verification status.

Existing work	Consequence for the present recommendation
Candidate scheduling	A generic scheduling induction is already proposed, and the current K217 change removes a seed-specific repetition.
Quotient unification	A recent proposal derives the QHyp interface from CP/GIC and reuses one later bisimulation-quotient development. That proposal is distinct from removing the *earlier* indiscernibility quotient.
Direct full seed	The sharing-interfaces packet explicitly sketches the direct Lean route but labels it uncompiled. The corresponding Rocq construction already appears in `rocq224/src/Complete222.v`. The contribution here is checked implementation and integration work, with those origins acknowledged.

The common design principle is to expose the strongest useful theorem that is already available, and then prove each reusable argument at the weakest interface it actually needs. The direct-seed result uses a stronger semantic interface than the old caller exploited. The propositional core uses a weaker syntactic interface than its old proof files suggested.

3 The Lean opportunity: use the original model

3.1 The current lower chain

The numerical calculus C_{222} derives the code 1 exactly when the relevant set theory is refutable. The difficult lower direction starts with a hypothetical model of the named formulas whose equality-interpreted numerical codes are derivable. In the previous route, the named seed bundle was first obtained only for equality-free matrices. The model was then quotiented by membership indiscernibility so that its new equality could be represented by an equality-free formula upstairs. A satisfaction-transport induction and representative choices extended the seed property to all named matrices.

This is a valid general construction. However, the actual completeness interface already supplies more than an arbitrary model of equality-free formulas.

3.2 The decisive completeness interface

Write $\mathrm{th}\, f$ for the membership-indiscernibility interpretation of equality, and $\mathrm{enc}\, f$ for the numerical encoding of a named formula. The theory used in the lower bridge is

$$\mathrm{DerivableNF}(f) \quad \Longleftrightarrow \quad \mathsf{C}_{222}(\mathrm{enc}(\mathrm{th}\, f)).$$

The syntactic premise of `KEind222.c222_complete` asks for derivability of $\mathrm{enc}(\mathrm{th}\, f)$ for theory members f. Its semantic premise, however, supplies

$$\forall f\ \big(\mathrm{DerivableNF}(f) \Longrightarrow \forall e\ \mathrm{Sat}(M, e, f)\big). \tag{1}$$

It supplies truth of f, with literal equality, on the original domain of M. It does not merely supply truth of $\mathrm{th}\, f$. This is the fact that permits the shorter route.

There is no circularity. The completeness theorem is proved using the named calculus and its interpretation; it does not import the seed-model or indiscernibility developments that this refactor bypasses.

3.3 The seed property

Let E be an arbitrary binary relation on a domain V. Write $x \subseteq_E y$ for $\forall w\,(E(w,x) \to E(w,y))$. For a binary predicate Q and an input A, the seed property is

$$\begin{aligned}\mathrm{Seed}_E(Q,A) \iff \exists C\,[&(\forall y\,(Q(A,y) \to \exists z\,(Q(A,z) \wedge E(z,C))))\\ &\wedge \forall y\,(E(y,C) \to \exists x\,[Q(x,y) \wedge\\ &\qquad \forall x'\,(x' \subseteq_E y \to \forall y'\,(Q(x',y') \to \exists z\,(Q(x',z) \wedge E(z,C))))])].\end{aligned}$$

Thus C answers any output request of A, and each element of C is an output whose subsets also have their requests answered inside C.

A full seed model is irreflexive and has this property for every named formula ψ, with the relation

$$Q_{\psi,e}(x,y) = \mathrm{Sat}(E, e[0 \mapsto x][1 \mapsto y], \psi).$$

Equality is permitted in ψ, and all other free names are interpreted as parameters by e.

4 A direct full-seed theorem

4.1 A fresh name supplied by the interpreted code

The repository's numerical constructors are

$$\begin{aligned}&\mathrm{enc}(R(i,j)) = 4P(i,j)+1, && \mathrm{enc}(i=j) = 4P(i,j)+3,\\ &\mathrm{enc}(\bot) = 1, && \mathrm{enc}(f \to g) = 4P(\mathrm{enc}\,f, \mathrm{enc}\,g),\\ &\mathrm{enc}(\forall i\,f) = 4P(i, \mathrm{enc}\,f)+2,\end{aligned}$$

where P is the pairing function already used by the machine development. The existing projection bounds give $x \le P(x,y)$ and $y \le P(x,y)$.

Lemma 4.1 (Free names are below the code)**.** *For every named formula f and natural number i,* $\mathrm{Free}(i,f)$ *implies* $i < \mathrm{enc}\,f$.

Proof. Induct on the formula. For membership and equality atoms, i is one of the two names, and the pairing projection bounds together with the positive tag give the inequality. Falsity has no free names. In an implication, apply the induction hypothesis to the subformula in which i is free, and then the corresponding pairing projection bound. In a universal formula, free occurrence implies free occurrence in its body, and the body-code projection bound applies. These are the five branches of `free_lt_enc`. □

Lemma 4.2 (Equality interpretation preserves free support)**.** *If* $\mathrm{Free}(i,f)$*, then* $\mathrm{Free}(i, \mathrm{th}\,f)$.

Proof. The relation, falsity, implication, and universal cases follow directly from the definition and induction. Equality $a = b$ is interpreted by the conjunction of agreement of incoming and outgoing membership. Its auxiliary quantified name is $a+b+1$, which differs from both a and b; consequently both original argument names remain free. This proves the equality case. The assertion concerns *free* occurrences, not every syntactic occurrence. □

For an arbitrary named matrix ψ, including equality, set

$$m_\psi = \neg\neg\psi, \qquad k_\psi = \mathrm{enc}(\mathrm{th}\,m_\psi).$$

The double negation gives $k_\psi > 1$ directly from the outer implication code and the right projection bound $1 \le P(\mathrm{enc}(\mathrm{th}\,\neg\psi), 1)$. Moreover, k_ψ is not free in m_ψ. If it were free, the two lemmas would give

$$k_\psi < \mathrm{enc}(\mathrm{th}\, m_\psi) = k_\psi,$$

a contradiction. Thus one and the same code supplies the enabled numerical bundle label and the freshness needed for the named semantic proof. No search for an extra fresh variable or alpha-renaming pass is required.

4.2 Named bundle semantics

The named syntax has genuine falsity, and its negation is implication into that falsity. Let $B(a, b, k; m)$ denote the named seed bundle `bundleNF`. Its request and closure subformulas are the existing `reqNF` and `closNF`. The exact names a, b, k remain explicit.

Theorem 4.3 (Named bundle equivalence)**.** *Suppose $a \neq b$, $k \neq a$, $k \neq b$, and k is not free in m. Then, for every relation E and environment e,*

$$\mathrm{Sat}(E, e, B(a, b, k; m)) \quad \Longleftrightarrow \quad \mathrm{Seed}_E\big((x, y) \mapsto \mathrm{Sat}(E, e[a \mapsto x][b \mapsto y], m), e(a)\big).$$

No irreflexivity hypothesis is required for this named-formula theorem.

Proof. First normalize the named request. Since $k \neq b$, it says exactly that every output of the fixed input has some output in the container named by k. Repeated assignments to b collapse by the environment-update identity.

Next normalize the closure subformula. Commuting updates at the distinct names a, b, k gives: every member y of the container is an output of some input x, and every subset x' of y has each output request answered in the container. This is `named_clos_iff`.

The bundle's auxiliary implication from a membership atom to its k-universal closure is valid because k differs from its two names. The outer negated universal implication is then converted, classically, to an existential container satisfying the closure and request conditions. Finally, freshness of k in m removes the container assignment from every occurrence of the matrix. The resulting conjunction is the definition of Seed_E, in reversed order. □

Remark 4.4 (Why the numerical theorem still needs irreflexivity). The decoded numerical code 1 is represented by a self-membership atom; irreflexivity makes it false. Named $\mathsf{NF}.\bot$, on the other hand, is interpreted as literal falsehood. Removing irreflexivity from the named theorem does not permit removing it from the numerical soundness theorem. In particular, the existing soundness development still covers arbitrary natural codes and coincident input/output roles.

4.3 Derivability of the named bundle

Equality interpretation commutes with the named bundle:

$$\mathrm{th}\, B(a, b, k; m) = B(a, b, k; \mathrm{th}\, m).$$

Every atom introduced by the bundle macros is a membership atom, so this identity requires no freshness side condition. With $k = k_\psi$, the existing encoding identity therefore gives

$$\mathrm{enc}\big(\mathrm{th}\, B(0, 1, k_\psi; m_\psi)\big) = \mathtt{bundle222}(0, 1, k_\psi).$$

Since $k_\psi > 1$, the numerical bundle constructor proves the right side derivable. Hence $B(0, 1, k_\psi; m_\psi)$ belongs to DerivableNF.

Theorem 4.5 (Direct full seed)**.** *Let E be an irreflexive relation satisfying every* DerivableNF *formula under every environment. Then* FullSeed(E) *holds on its original domain.*

Proof. For arbitrary ψ, e, apply the model hypothesis to the derivable named bundle just constructed. Theorem 4.3, the bound $k_\psi > 1$, and freshness give $\mathrm{Seed}_E(Q_{m_\psi,e}, e(0))$. Classical double-negation elimination gives pointwise equivalence of $Q_{m_\psi,e}$ and $Q_{\psi,e}$. The existing `seedProp_congr` transports the seed property across this pointwise equivalence. Combining these instances with the supplied irreflexivity gives FullSeed(E). □

This is precisely the new direct semantic component. The older theorem concludes FullSeed(qmem E) on a quotient domain; the direct theorem concludes FullSeed(E). The argument does not assert that ψ and $\mathrm{th}\,\psi$ have the same truth value on the original domain. The interpreted formula is used to prove derivability and choose the private name; the model hypothesis evaluates the original named bundle.

5 Integration into the Lean lower bridge

5.1 What can now be removed from this proof path

The old path needs the relation `Ind`, its setoid, the quotient `IndQ`, lifted membership `qmem`, equality and surjectivity facts for representatives, a satisfaction-transport induction, and a transfer of seed containers through quotient representatives. None of these is needed by the direct lower bridge.

The later set-theoretic construction still needs its tagged domain, bisimulation quotient, and well-founded part. Those solve a different mathematical problem. The proposed CP/GIC-to-QHyp adapter can be considered separately; this report's direct seed result neither assumes its adoption nor claims its savings.

Stage	Previous lower bridge	Direct lower bridge
Completeness	Model of original derivable named formulas	Same model
Seed instances	Equality-free matrices first	All named matrices directly
First quotient	Membership indiscernibility and satisfaction transport	Not used
Seed model	Quotient domain	Original domain
ZF construction	Tagged domain, bisimulation quotient, well-founded part	Same construction
Witness	Quotient class of $e(0)$	$e(0)$

5.2 A small interface is essential

Simply importing the old quotient module to obtain its `FullSeed` declaration would retain the entire old route. The implementation therefore separates the shared named bundle definitions, the theory predicate, and the full-seed specification from the optional quotient proof. The downstream `KLowerSeed` module needs only this specification. Existing quotient clients can continue to use the legacy module and its original theorem statements.

This is a compatibility-preserving import cut, not deletion of a useful general quotient theorem from the repository. The exact overlay and source census record which modules leave the K215 closure and which declarations remain available to other clients.

5.3 The endpoint substitution

At the mathematical boundary, the lower bridge changes to the following shape:

```
have hfull := fullSeed_direct_of_derivable mem hirr hT
exact zf_consistent_of_formSeed
  (formSeed_of_fullSeed hfull) hfull.irr (e 0) hf
```

The source calculus, its code 1, the definition of finite ZF refutation, the ZF/ZFC refutation equivalence, and the physical-machine theorem type are unchanged. In particular, this argument does not add an inhabited-domain axiom: the environment already supplies the witness $e(0)$.

Removing a semantic quotient does not imply removing `Quot.sound` from Lean's printed axiom set. Existing classical reasoning and extensionality infrastructure still use Lean's standard principles. The relevant check is that the exact reported axiom profile is preserved and no `sorryAx`, new axiom, or unchecked native-computation principle enters.

6 The Rocq opportunity: one propositional derivation library

6.1 The duplicated fragment

The pinned `ZF.L1_LProp` file and the K215-local `L224Prop` file contain 71 corresponding lemmas: 59 intermediate derivations and 12 public propositional rules. An independent normalization check finds identical statements and proof bodies after renaming the theorem predicate, the axiom constructors, modus ponens, and the exported prefixes.

The two full named calculi should not be identified. They have different quantifier/vacuity interfaces. The repeated *propositional* derivations, however, use only implication, negation, modus ponens, and three rules common to both calculi.

6.2 The minimal interface

Let F be any type of formula objects, with operations $\Rightarrow$ and $\neg$, and predicate Pr. Require

$$\begin{aligned}
\text{MP}: &\quad \Pr(a \Rightarrow b) \to \Pr(a) \to \Pr(b),\\
\text{B1}: &\quad \Pr\big((a \Rightarrow b) \Rightarrow ((b \Rightarrow c) \Rightarrow (a \Rightarrow c))\big),\\
\text{B2}: &\quad \Pr\big((\neg a \Rightarrow a) \Rightarrow a\big),\\
\text{B3}: &\quad \Pr\big(a \Rightarrow (\neg a \Rightarrow b)\big).
\end{aligned}$$

The Rocq module type `PropositionalBasis` expresses exactly these data. The functor `PropositionalRules` contains one copy of the inherited derivations. No syntax induction, quantifier constructor, equality atom, or set-theoretic axiom is part of this contract.

Three original proof bodies used the arbitrary formula `Weq 0 0`, although their statements did not mention it. Each lemma already binds a formula `w0`; substituting that formula for the arbitrary choice removes even the need for a distinguished formula or an inhabited-carrier assumption. The core therefore applies uniformly to an arbitrary formula type, including an empty one.

6.3 Why the abstraction is sound

Every original derivation is a finite term constructed from earlier derivations, MP, and the three basis rules. Replacing the concrete symbols with interface fields yields the same well-typed proof

term over arbitrary interface data. This is the mathematical content of the abstraction. Kernel compilation checks each such term; the code does not assume a broad soundness principle for the full named calculus.

For the ordinary calculus, instantiate the interface using `lprov`, `LDET`, and `LB1`–`LB3`. For the restricted calculus, instantiate it using `lprov224`, `MDET`, and `MB1`–`MB3`. Both use their original implication and negation constructors. The compatibility facades retain the original public and helper names.

The module-type fields are parameters of a functor, not new global axioms of the concrete endpoints. Their implementations are the constructors already present in the original inductive derivability relations. The concrete assumption reports must therefore be closed; this is checked explicitly.

6.4 The real size tradeoff

For the two concrete clients together, the old source occupies **297 lines and 23,160 bytes**. The shared core plus its two compatibility facades occupies **261 lines and 16,001 bytes**: a reduction of **36 lines and 7,159 bytes**, approximately **30.9%** of those source bytes. These figures include the new abstraction and both facades, but exclude audit clients and documentation from both sides of the implementation comparison.

Adopting only the restricted-calculus facade while retaining the complete old ordinary propositional development would *increase* the selected source by 97 lines and 3,653 bytes. The paired adoption is the actual simplification. The report does not infer a proof-object-size reduction or a clean-build speedup from these source figures; those are separate measurements.

7 Verification, reproduction, and adoption

7.1 Evidence is tied to source identities

The archive includes the repository pin, source SHA-256 values, the proposed overlays, compiler logs, assumption reports, typed API checks, and source metrics. The Lean closure census is independent of the compiler and labels its output as a source measurement. The Rocq evidence records the selected component and its upstream references. No old proof receipt is treated as a certificate for newly combined source bytes.

7.2 Lean verification

The proof compiler is Lean 4.32.0, commit `8c9756b28d64dab099da31a4c09229a9e6a2ef35`, matching the repository's toolchain. The direct bundle theorem, the free-name bound, preservation of free support through equality interpretation, and the direct seed theorem have been compiled against the reviewed source. The successful named-bundle and direct seed reports use only `propext`, `Classical.choice`, and `Quot.sound`.

The final clean replay compiles 112 project modules serially with no project object reuse. Every compilation succeeds. The two typed audit clients check the generic endpoint and both direct/legacy seed APIs, producing eleven expected axiom reports. The two free-support lemmas use `propext` and `Quot.sound`; the other nine audited results use the standard three axioms. There are no warnings in the changed modules or audit clients; 18 preexisting linter warnings occur in unchanged dependencies. The recorded elapsed time is 113.52 seconds for this replay, with no baseline speed comparison implied.

The exact compilation records are described in `lean/README.md` and its accompanying evidence. The interface split is checked separately from the optional old quotient route, so retaining old clients cannot silently reintroduce that route into the new K215 import closure. The source-only census computes the full hypothetical K215 closure of the overlay, including its unchanged physical certificate modules; it does not compile those certificates.

7.3 Rocq component verification

The local component check uses Coq 8.18.0 with OCaml 4.14.1. Ten selected source modules are compiled serially, including the shared core, both actual derivability instances, both compatibility facades, and the unchanged `ZF.L1_LMeta` and `K215F.L224Meta` consumers. The audit restates all 142 original theorem signatures. Its 26 printed assumption reports are closed: 12 public propositional rules in each calculus and two meta-theory results. Recursive `coqchk` succeeds for the selected consumers and their imports.

The only compatibility edit in the old meta-theory consumer for this local check is the import-prefix spelling `Stdlib` to `Coq`. The packet does not claim that Coq 8.18 is the production toolchain: the maintained K215 Rocq lane pins Rocq 9.1.0 and stdlib 9.0.0. The supplied component must be registered in that driver and checked there before adoption. Raw exit codes, source and object hashes, API logs, and the recursive kernel-check record are retained in `rocq/evidence/`.

7.4 Independent review and limits of performance claims

An independent source-and-receipt review verifies the direct argument's completeness premise, the exact import counts, the moved interface statements, the 142 Rocq API signatures, ten Rocq source/object records, and the closed assumption reports. The final Lean replay has its own 112 bound module records. This review is labeled as a review, not a second compiler run. Its detailed findings are included with the archive.

The evidence contains per-module elapsed times to reproduce the work's scope. These are not a controlled before/after build benchmark. No clean-build speedup or proof-object-size reduction is claimed. Module counts, physical source lines, source bytes, and number of maintained derivation bodies are kept as separate metrics.

7.5 Suggested integration order

1. Apply the Lean interface split and direct-seed component to the pinned source in an isolated checkout. Compile the direct theorem, its semantic lower bridge, and the retained legacy interfaces. Compare exact theorem types and printed axioms.

2. Run the repository's K215 endpoint verification driver on the changed closure. Use its exact-source provider reuse policy for unchanged physical modules; do not relabel a source census as a successful machine build.

3. Adopt the Rocq core and both facades together in the source-staging pipeline. Compile the exact production namespace and consumers under its pinned Rocq/stdlib versions, and run the corresponding recursive kernel check.

4. Update generators or staging rules so regeneration reproduces the simplified sources. Retain the derivative-file attribution and upstream pins. A locally edited generated file alone is not a durable integration.

5. Record separate results for logical API preservation, source reduction, proof objects, and build time. Only then replace the currently approved endpoint provider records.

The work delivered here is a proposal package for this sequence. It does not modify the public repository or ask for permission to publish changes. The user requested an archive for later incorporation.

8 What should not be simplified away

Several tempting reductions would change the theorem rather than simplify its proof.

Arbitrary numerical codes. The machine manipulates arbitrary natural formula codes. Restricting a numerical soundness theorem to neatly decoded formulas would require a new reachability invariant. The direct lower bridge has no such consequence for the upper bridge.

Coincident role names. The ordinary named-bundle theorem uses distinct input and output names. The numerical soundness theorem separately handles degenerate roles. Its diagonal case must remain; the direct construction simply selects the fixed distinct roles 0 and 1 for its own use.

The later set-theoretic quotient. The preliminary indiscernibility quotient enforces an equality interpretation. The later bisimulation quotient belongs to the ZF model construction. Removing the first does not justify removing the second.

Progress in machine simulations. The present proposal does not alter the physical runtime simulation. Existing positive-step obligations prevent a target halt from being hidden by an infinite sequence of zero-duration source stutters. A future runtime compression must preserve those obligations or supply an independently proved well-founded rank.

Exact statements and assumptions. A shorter lemma with a stronger unproved premise is not a replacement for the closed endpoint. Explicit typed clients and concrete assumption reports are therefore part of the deliverable, not optional decoration.

9 Further research questions

The next questions are concrete enough to have measurable success criteria.

1. **Can the named and numerical bundle proofs share one normalization argument?** Both normalize requests, closure, and a private marker. An abstract matrix-satisfaction interface could expose *semantic* freshness and update laws once, with named and decoded instances. Semantic freshness means that updating the private name does not change matrix truth. Requiring only syntactic absence of a free name would be too restrictive for the numerical code-zero case: its decoded tautology can mention name zero while its truth remains independent of that name. The existing `sat_dec_update` and the new `sat_update_fresh` already supply the two semantic interfaces. The target is a net deletion after including the interface, with the numerical diagonal case and irreflexivity assumptions still explicit.

2. **How far can the Rocq basis be reused beyond these 71 lemmas?** The unchanged meta-theory consumers already compile through the facades. A next step is to factor only the common contextual/deduction operations, while leaving the differing quantifier/vacuity hypotheses as parameters. The success criterion is paired source reduction plus preserved original signatures, not a claim that the two full calculi coincide.
3. **Can the direct lower bridge and the existing QHyp quotient adapter be adopted together?** They address different stages and should compose. The combined closure must be built afresh or through exact bound providers; source savings from two independent proposals cannot simply be added when they share imports or wrappers.
4. **Which proof-interface cuts improve build cost as well as source readability?** Measure clean and incremental builds with identical toolchains and resource settings. Keep generated certificate data, mathematical proof source, and compiled proof-object sizes separate.
5. **Can more cross-assistant architecture be aligned?** The Rocq direct seed construction suggested the Lean simplification; the Lean practice of shared named-calculus interfaces suggested the Rocq factoring. A theorem-interface correspondence could prevent future independent copies of essentially the same argument, without forcing both assistants to use identical encodings or proof tactics.
6. **Can the internal consistency statement be strengthened without duplicating the machine proof?** The internal PA endpoint is a distinct research problem. Isolate the additional formalized proof-translation and arithmetization obligations at the existing semantic/calculus boundary; do not rebuild a second physical runtime proof merely to change the metatheoretic presentation.

10 Sources and attribution

All ProveIt URLs below refer to the immutable reviewed commit, not the moving `main` branch. Long file paths are given so the report remains usable after incorporation into the repository.

[1] K215 README. Exact machine, theorem statements, evidence routes, and formal boundary.

[2] Fourteenth K215 proof-simplification checkpoint. Maintained source counts, checked scopes, and assumption profiles.

[3] Proof-simplification proposals index. Earlier source, runtime, scheduling, quotient, and interface proposals.

[4] Direct full seed proposal. Prior uncompiled construction implemented by the Lean component here.

[5] `KEind222.lean`; `KSeedModel222.lean`; `KIndisc222.lean`; `KEndpoint.lean`. Exact old lower-chain interfaces and proofs.

[6] `rocq224/src/Complete222.v`. Existing Rocq fresh-code construction and direct seed route.

[7] Jeff Ketchersid's pinned busy-beaver source and Elliot Glazer's pinned K230 Rocq source. The inherited named propositional derivations, source calculi, and set-theoretic construction retain their original authorship and terms.

The abstract propositional derivation bodies are adaptations of the existing Jeff Ketchersid proofs. The seed principle and lower construction are inherited from the Glazer/K-family development. This packet's contribution is the checked refactoring, concrete adapters, direct Lean implementation, integration overlay, exact accounting, and review; it does not claim discovery of these underlying logical principles. Notices and source identities accompany the code.

A Archive map and reproducibility

Path	Contents and role
`report/`	This article, its LaTeX inputs, and the compiled PDF.
`lean/`	Direct-seed source overlay, compatibility clients, reproduction program, patch, source identities, and compilation evidence.
`rocq/`	Shared propositional core, two exact facades, 142-signature audit, pinned input snapshots, deterministic producer, reproduction program, source patches, notices, and compiler/kernel evidence.
`tools/closure_census.py`	Read-only Lean import and source-size census; accepts an explicit checkout and repository-relative or module-root overlays.
`evidence/`	Repository pin, baseline and proposed closure measurements, independent review, and delivery manifest.
`README.md`	Entry point, exact validation scope, integration instructions, and instructions for rebuilding this article.

To rebuild the PDF, run the following from `report/` in an environment with pdfLaTeX and the listed standard packages:

```
pdflatex -interaction=nonstopmode -halt-on-error main.tex
pdflatex -interaction=nonstopmode -halt-on-error main.tex
```

The reproduction programs do not require compiled proof objects from this archive. Proof-object files and compiler installations are deliberately omitted; the formal source, inputs or exact retrieval pins, and logs are provided.